

AI-ENHANCED IoT SOLAR MONITORING SYSTEM

¹Ruvarashe Machona ²C.C. Muchenjekwa

¹h220047m@hit.ac.zw ²ccmuchenjekwa@hit.ac.zw

Harare Institute of Technology, Harare, Zimbabwe

Department of Information Technology, Harare Institute of Technology

Abstract—

The integration of AI and IoT facilitates continuous monitoring, prediction, and fault reporting of solar photovoltaic (PV) panels. However, at the Harare Institute of Technology (HIT), monitoring processes are still manual and reactive, resulting in delayed detection of anomalies, low efficiency in energy production, high cost, and increased wear-and-tear on equipment. The following research provides details regarding the design and implementation of an AI-powered IoT solar monitoring prototype designed specifically for HIT. The proposed system involves capturing multi-dimensional telemetry data using an ESP32 sensor node and relaying the data collected back to the backend (Next.js). Subsequently, the backend validates the data, stores it in a database (SQLite using Prisma), and runs inference using the ONNX model to generate predictions such as efficiency and its change, potential fault probability, fault type, and health score of the panel among others. A live dashboard is used to visualize the output, along with historical analysis. In order to achieve prompt responses to faults, the backend automatically sends emails and notifications to WhatsApp when critical points or model outputs suggest that an issue might be present.

Index Terms— Artificial Intelligence (AI), Internet of Things (IoT), solar photovoltaic (PV) monitoring, ESP32, time-series telemetry, ONNX Runtime, machine learning inference, Random Forest, Gradient

Boosting, efficiency prediction, fault detection, fault classification, health score, predictive maintenance, dashboard visualization, SQLite, Prisma, email alerts, WhatsApp alerts.

I. INTRODUCTION

The use of Solar PV systems has been growing, aiming to minimize dependence on the grid and lower costs for operations. Nevertheless, PV output depends on environmental factors (changes in irradiation levels, soiling effects, shadowing, and heat), as well as electrical issues (loosely connected parts and damaged wiring). Traditional monitoring techniques like periodic inspections and deterministic threshold-based alarms do not work well in identifying progressive performance deterioration, resulting in continuous energy wastage before any action is taken. AI-based monitoring systems could be able to predict nonlinear PV behavior and detect anomalies from noisy data streams [1], [2].

In this work, we describe a proof-of-concept design for an AI-based solar monitoring platform for HIT, which utilizes cheap sensors, trusted telemetry ingestion, local storage for provenance purposes, machine learning models running using ONNX Runtime, a web-based visual interface, and email/WhatsApp alerts. Our contributions are: (1) the development of a scalable and deployable monitoring pipeline; (2) the construction of a model stack that allows us to accurately estimate efficiency and diagnose faults; and (3) a reliable alerting

mechanism according to institutional communication requirements [6], [7].

II. LITERATURE REVIEW

Various AI techniques have been utilized for photovoltaic modeling, prediction, and monitoring over several decades now. Mellit and Kalogirou review various AI techniques implemented in PV systems and point out the strengths of data-driven models in describing non-linear relationships between PV system outputs and environmental factors [1]. Fault detection and diagnostics (FDD) continue to be an active research subject since faults have direct consequences for PV system output and might even pose safety hazards. The importance of incorporating both electrical variables (V, I, P) and environmental variables (irradiation and temperature) into fault detection is pointed out in reviews [2].

Fault detection algorithms based on machine learning demonstrate excellent results in varying operating conditions. For example, Garoudja et al. confirm that ML classifiers can diagnose faults in PV systems despite different meteorological conditions by employing proper feature representation [5]. It should also be mentioned that operational readiness is a matter of importance when applying AI solutions for FDD. Indeed, a deployed system should not only deliver reliable predictions but also offer high quality data ingestion, logging, and interpretation capabilities [6].

In IoT implementations, latency and robustness can be enhanced by processing closer to the source of the data or using hybrid models. Fog/edge computing enables cloud computing extensions to be made closer to the edge of the network, facilitating low-latency stream analysis appropriate for real-time monitoring and alarms [3], [4]. The proposed solution utilizes a modular design consistent with edge-cloud computing but practical to implement at the prototype level.

TABLE I. COMPARATIVE SUMMARY (COMMERCIAL vs. PROTOTYPE)

Sys tem	Granu larity	AI/Pr edicti on	Faul t Dia gno sis	Alert ing	Depl oym ent Focu s
Sol ar- Log / SM A / Enp has e	String /Mod ule (vend or)	Limit ed/Ru le- based	Basi c thre shol ds	Emai l/app notifi cation s	Repo rting matu rity
Res earc h ML Sys tem s	Varie s	High (ML/ DL)	Adv anc ed mod els	Often exper iment al	Mod el accur acy
Thi s Prot oty pe (HI T)	Senso r- level	RF + GBC via ONN X	Faul t pro b + type	Emai l + What sApp	Depl oyab le end- to- end

III. SYSTEM/SOLUTION DESIGN

The system is organized as a modular pipeline: sensor acquisition → telemetry transport → backend validation → persistence → ML inference → visualization → alerts. Modularity reduces coupling and supports future scaling (multiple panels/sensor nodes) because changes to one layer do not require redesign of the full stack [3], [4].

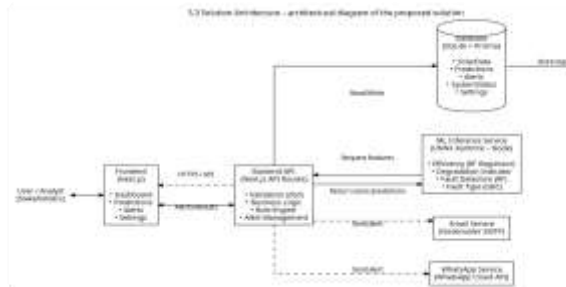


Fig. 1. Solution architecture (ESP32 → Next.js API → SQLite/Prisma → ONNX inference → Dashboard → Email/WhatsApp).

A. Context and Data Flow

The system will be interacted with by the stakeholders, which include technicians and energy office users, using the web dashboard and communicating proactively about any abnormal conditions using the alert messages. The telemetry data is submitted to the backend endpoint where it is validated, stored, and analyzed.

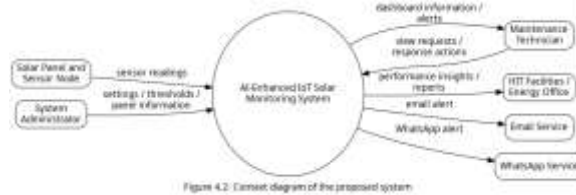


Fig. 2. Context diagram of the proposed system (including email and WhatsApp alert channels).

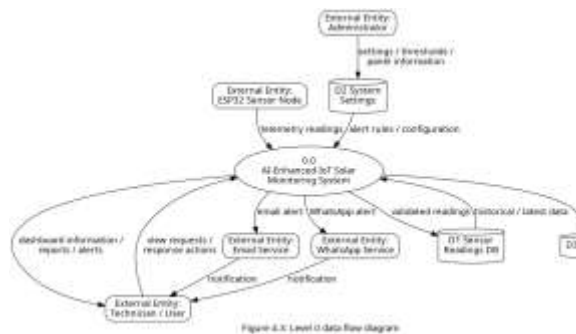


Fig. 3. Level 0 data flow diagram.

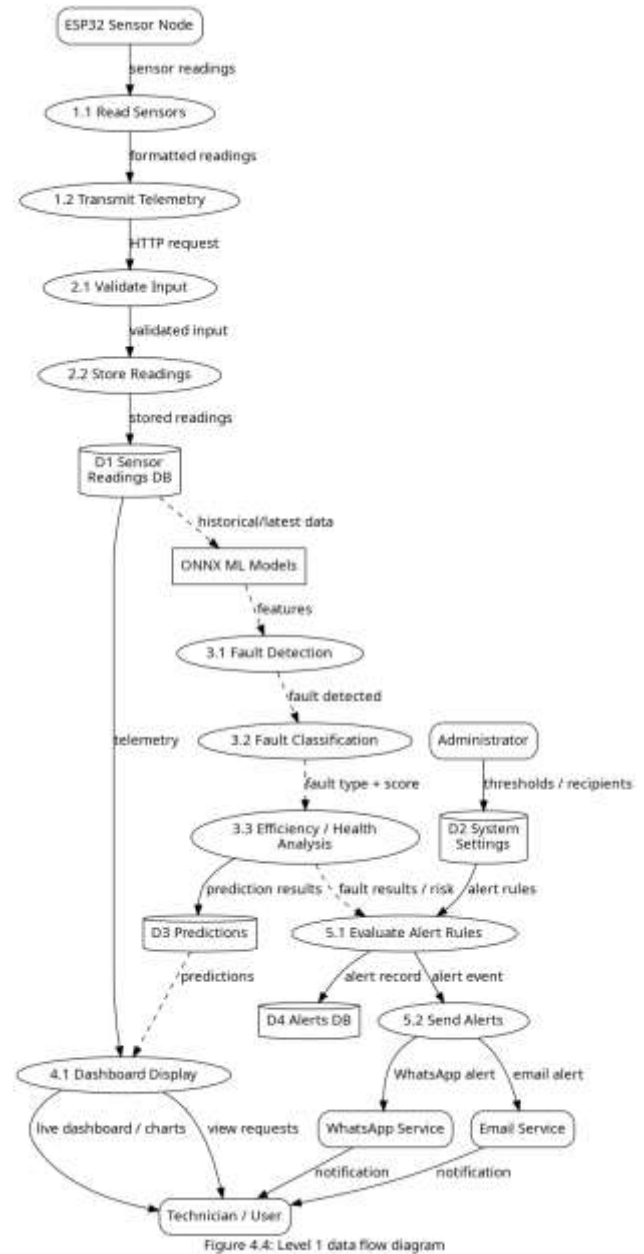


Fig. 4. Level 1 data flow diagram.

B. Database Model

The database allows for persistent storage of the time series data from solar telemetry and system results. The basic entity is the SolarData that consists of measurements made repeatedly throughout the period. The Status of Systems keeps information about the connectivity status, whereas the Prediction keeps the output of models. The

Alert stores the messages displayed in the Alerts page [6].

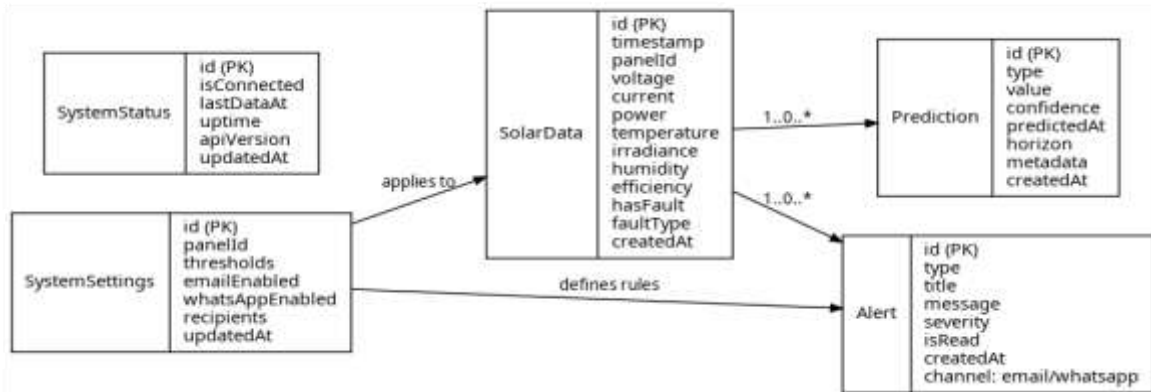


Fig. 5. Entity-relationship diagram (logical relationships for telemetry, predictions and alerts).

C. Algorithm Design

The prediction pipeline computes derived features such as power ($P = V \times I$), then applies ensemble models. Random Forest regression is used for efficiency prediction due to robustness to noise and non-linear feature interactions. Random Forest classification is used for fault detection, and Gradient Boosting classification is used for fault type identification to improve separation among similar fault classes [1], [5], [7]. Confidence is reported as 0–100% to communicate uncertainty to stakeholders.

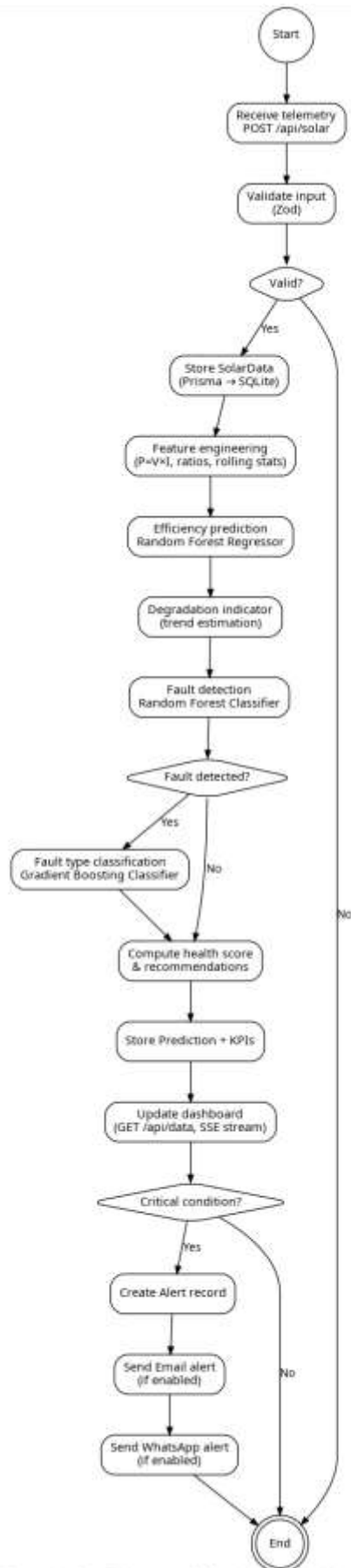


Figure 5.3: Processing Flow (Telemetry → AI Predictions → Dashboard → Alert)

Fig. 6. Processing flow (telemetry → features → models → health analysis → dashboard/alerts).

IV. IMPLEMENTATION

The prototype is implemented with three integrated components: (i) ESP32 firmware for acquisition and HTTP posting, (ii) Next.js backend for validation, persistence, and ONNX inference, and (iii) a dashboard for visualization and alert management. SQLite via Prisma provides local persistence suitable for prototype deployments, while ONNX Runtime enables portable server-side inference without requiring a Python service at runtime.

A. Telemetry Ingestion

- ESP32 reads voltage (via divider), temperature/humidity (DHT22), and irradiance proxy (LDR), then posts JSON to POST /api/solar.
- Backend validates incoming telemetry before processing; invalid payloads are rejected to preserve database integrity.
- Derived fields such as power and efficiency indicators are computed consistently for storage and visualization.

B. Model Integration

- Efficiency prediction: Random Forest Regressor (non-linear regression on tabular telemetry features).
- Fault detection: Random Forest Classifier (binary classification: normal vs. fault).
- Fault type: Gradient Boosting Classifier (multi-class: soiling, shading, connection, aging, etc.).
- Models exported to ONNX and executed in the backend to keep inference lightweight and deployable.

C. Interface Modules

The user interface is organized into four modules: Dashboard, Predictions, Alerts, and Settings. The Dashboard provides live cards and historical charts. The Predictions module shows health score, efficiency forecast, degradation cues, and maintenance

recommendations. Alerts logs warning/error events. Settings provides panel configuration, notification controls, and API information for integration.



Figure 5.6.1: Dashboard Wireframe (Status + KPI Cards)

Fig. 7. Dashboard wireframe (status + KPI cards).

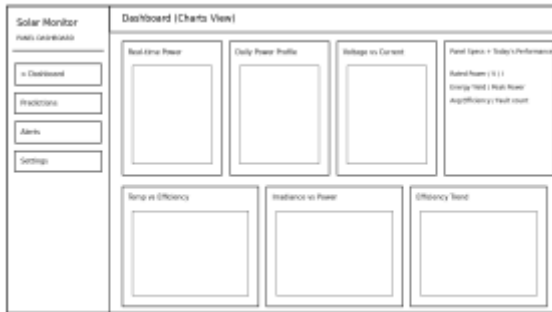


Figure 5.6.2: Dashboard Wireframe (Charts + Performance Summary)

Fig. 8. Dashboard wireframe (charts + performance summary).



Figure 5.6.3: Predictions Wireframe (Health Score + Efficiency Forecast)

Fig. 9. Predictions wireframe (health score + efficiency forecast).



Figure 5.6.4: Predictions Wireframe (Fault Probability + Degradation Rate) - Clear Version

Fig. 10. Predictions wireframe (fault probability + degradation rate).

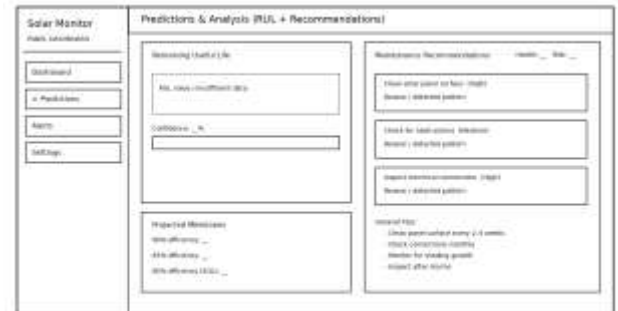


Figure 5.6.5: Predictions Wireframe (RUL + Maintenance Recommendations)

Fig. 11. Predictions wireframe (RUL + maintenance recommendations).

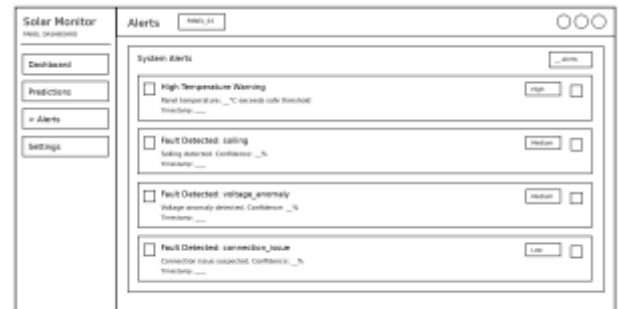


Figure 5.6.6: Alerts Wireframe (System Alerts List)

Fig. 12. Alerts wireframe (system alerts list).



Figure 5.6.7: Settings Wireframe (Panel + Connection Configuration)

Fig. 13. Settings wireframe (panel + connection configuration).

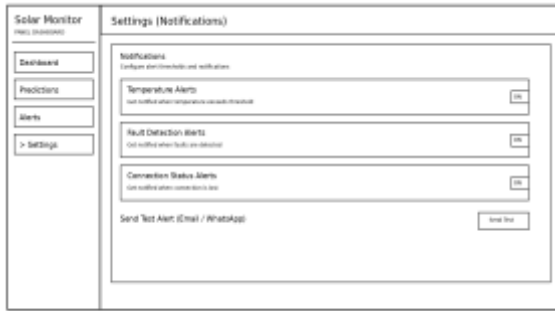


Figure 5.8.8: Settings Wireframe (Notifications)

Fig. 14. Settings wireframe (notifications).



Figure 5.8.9: Settings Wireframe (API Information + ESP32 Snippet)

Fig. 15. Settings wireframe (API information + ESP32 snippet).

V. RESULTS AND EVALUATION

Evaluation combined functional verification (end-to-end workflow) and non-functional checks (responsiveness and stability). Tests covered telemetry ingestion, validation, storage, dashboard updates, prediction visualization, alert logging, and delivery of notifications over email and WhatsApp.

A. Model Metrics (Prototype Training/Validation)

Model	Task	Metric (s)	Observed	Notes
RF Classifier	Fault detection	Accuracy; F1	98%; 0.86	High performance under simulated faults
GBC	Fault type	Macro-F1	≈0.80	Some rare classes harder

				to separate
RF Regressor	Efficiency prediction	R ² ; MAE	0.98; 0.53	Stable under noise; non-linear fit

B. Functional Test Cases

Test ID	Feature	Scenario	Expected	Actual	Result
T-C-01	Ingestion	Valid POST /api/solar	200 OK	200 OK	Pass
T-C-02	Validation	Missing voltage	400 reject	Rejected	Pass
T-C-03	Storage	10 readings insert	10 rows stored	Confirmed	Pass
T-C-04	Dashboard	Open dashboard	Latest visible	Visible	Pass
T-C-05	Prediction	Open predictions	Outputs shown	Shown	Pass
T-C-06	Alert log	Trigger high temp	Alert created	Created	Pass
T-C-07	Email	Critical condition	Email delivered	Delivered	Pass
T-C-08	WhatsApp	Critical condition	Message delivered	Delivered	Pass

C. Non-Functional Observations

- Responsiveness: dashboard updates observed shortly after new records were stored at the configured send interval.
- Reliability: continuous operation under periodic posting without crashes in the prototype environment.
- Security (prototype): notification credentials kept in environment configuration rather than hard-coded.
- Maintainability: modular separation allowed independent changes to firmware, backend, models and UI.

VI. DISCUSSION

The system highlights how low-cost telemetry from IoT together with deployable ML inference allows for practical monitoring of institutional PV facilities. The persistence of telemetry and prediction allows for reasoning on trends and explainability based on confidence and health scoring. Dual notifications via email and WhatsApp increase chances of an event being detected. The drawback in using remaining useful life and degradation is that they only become accurate after gathering longer timeseries data since shorter periods of observation make predictions less certain, and thus the system provides confidence scoring percentage values along with alerts of lacking data where necessary [6].

Next steps include calibration of irradiance sensing, improving the quality of current sensing hardware, collecting data sets for a longer period of time at HIT, and adding a light-weight gateway/node to enhance resilience and decrease reliance on connectivity, adhering to the principle of fog/edge computing [3], [4].

VII. CONCLUSION

The technical realization of the AI-driven IoT system designed for HIT was described in this paper. The proposed prototype is able to gather telemetry signals in real time, validate and store them, conduct ML predictions based on the ONNX framework, visualize outputs via a web interface, and

notify users via email and WhatsApp messaging services. Such a technical realization allows shifting from the current reactive approach to more proactive maintenance through early fault detection and accurate performance estimation.

REFERENCES

- [1] A. Mellit and S. A. Kalogirou, "Artificial intelligence techniques for photovoltaic applications: A review," *Progress in Energy and Combustion Science*, vol. 34, no. 5, pp. 574–632, 2008.
- [2] A. Massi Pavan, A. Mellit, and D. De Pieri, "Fault detection and diagnosis of photovoltaic systems: A review," *Renewable and Sustainable Energy Reviews*, vol. 81, pp. 314–327, 2018.
- [3] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. MCC Workshop on Mobile Cloud Computing (MCC '12)*, 2012, pp. 13–16.
- [4] A. V. Dastjerdi and R. Buyya, "Fog computing: Helping the Internet of Things realize its potential," *Computer*, vol. 49, no. 8, pp. 112–116, 2016.
- [5] E. Garoudja, A. Chouder, K. Kara, and S. Silvestre, "An enhanced machine learning based approach for failures detection and diagnosis of PV systems," *Energy Conversion and Management*, vol. 151, pp. 496–513, 2017.
- [6] T. Zonta et al., "Predictive maintenance in the Industry 4.0: A systematic literature review," *Computers & Industrial Engineering*, vol. 150, p. 106889, 2020.
- [7] M. L. D. Wong, C. Tan, C. H. Lim, and N. A. S. Hamzah, "Photovoltaic system fault detection and diagnosis using machine learning techniques," *IEEE Access*, vol. 8, pp. 149672–149685, 2020.
- [8] M. Hasan, A. Siddique, and T. Ahmed, "IoT-based real-time solar energy monitoring using ESP32," *IEEE Access*, 2022.

[9] Espressif Systems, “ESP32 ADC Documentation,” 2024.

[10] Microsoft, “ONNX Runtime Documentation,” 2026.

[11] Nodemailer, “Nodemailer Documentation,” 2026.

[12] Meta, “WhatsApp Cloud API Documentation,” 2026.